

CNN & Image Features - Key Recall Notes

Compact revision sheet: kernels, filters, classical features, weights, bias and Batch Normalization.

Big picture: Traditional computer vision uses hand-designed features such as SIFT, HOG and color histograms. CNNs learn their own features automatically by learning filter weights from data.

1. Kernel and Filter in a CNN

Kernel

A small matrix of learnable weights, e.g. 3×3 , that slides across the input and performs convolution.

Filter

The complete set of kernels that produces one output feature map. For multi-channel input, one filter spans all input channels.

$$\text{output at a location} = \text{sum}(\text{input patch} \times \text{kernel weights}) + \text{bias}$$

- A filter responds strongly when its learned pattern is present.
- Early layers often learn edges, color contrasts and simple textures.
- Deeper layers combine these into shapes, parts and object-level patterns.
- **Number of filters = number of output feature maps/channels.**

Recall: Kernel = small weight matrix. Filter = one feature detector producing one feature map. In casual usage, the words are often used interchangeably.

2. SIFT, HOG and Color Features

Feature	What it captures	Core idea	Typical use
SIFT	Distinctive local keypoints	Find interesting points, then describe the local neighborhood around each point.	Image/object matching, panorama stitching
HOG	Edges and shape	Count how frequently edges point in different directions inside local cells.	Pedestrian/object detection
Color	Distribution of colors	Represent how much of different colors occur, often using RGB/HSV/Lab histograms.	Tasks where color is informative

Recall: SIFT = keypoints. HOG = edge directions. Color = color distribution.

3. Traditional Features vs CNN Features

Traditional computer vision	CNN
Human chooses feature extractor: SIFT/HOG/color.	Network learns useful features from training data.
Feature design and classifier are often separate.	Feature extraction and classification can be trained end-to-end.
Often useful for specific tasks and smaller datasets.	Usually more powerful with sufficient data and compute.

4. Weights in Deep Learning / CNNs

Weights are the main learnable numbers of a neural network. They determine how strongly each input contributes to an output.

$$y = wx + b$$

- Large positive weight: strong positive influence.
- Negative weight: reverses the direction of influence.
- Weight near zero: input contributes little.
- In CNNs, the entries inside kernels/filters are weights.

Why weights matter in CNNs

The learned weight pattern determines **what feature a filter detects**. During training, backpropagation and the optimizer repeatedly adjust weights to reduce the loss.

random weights -> prediction -> loss -> backpropagation -> weight update

Recall: Learning in a neural network is largely the process of finding useful weight values.

5. Weight vs Bias

	Weight (w)	Bias (b)
Role	Controls how strongly an input affects the neuron.	Adds a learnable offset to shift the neuron response.
Equation	$y = wx + b$	$y = wx + b$
Line analogy	Controls slope.	Controls intercept / shift.
When $x = 0$	Weighted contribution becomes 0.	Output can still be b.
In CNN	Kernel values determine detected pattern.	Shifts activation of the resulting feature map.

Recall: Weight = **how much**. Bias = **how much to shift**.

6. Batch Normalization (BatchNorm)

BatchNorm normalizes intermediate activations during training so optimization is generally more stable and easier.

$$\hat{x} = (x - \text{batch mean}) / \sqrt{\text{batch variance} + \text{epsilon}}$$

After normalization, BatchNorm applies two learnable parameters:

$$y = \gamma \hat{x} + \beta$$

- **gamma** = learnable scale
- **beta** = learnable shift
- **epsilon** = tiny constant to avoid division by zero

Typical CNN ordering

Convolution -> BatchNorm -> ReLU -> next layer

Why BatchNorm is useful

- Helps stabilize training and can support larger learning rates.

- Reduces sensitivity to parameter initialization.
- Sometimes adds a mild regularization effect.

Training vs inference

Training	Inference / Testing
Uses statistics from the current mini-batch.	Uses running estimates of mean and variance accumulated during training.

Recall: BatchNorm = **Normalize -> Scale -> Shift.**

7. 30-Second Final Recall

Term	Remember this
Kernel	Small learnable weight matrix sliding over an input.
Filter	Feature detector producing one output feature map.
SIFT	Local keypoints + descriptors; robust to scale and rotation.
HOG	Histogram of edge orientations; captures shape.
Color feature	Represents color distribution, often via histogram.
Weight	Controls input influence; values learned during training.
Bias	Learnable offset that shifts a neuron response.
BatchNorm	Normalize activations, then learn scale gamma and shift beta.